



ปริญญานิพนธ์

INTELLIGENT SERVER OPERATIONS PLATFORM

ระบบปฏิบัติการและบริหารจัดการเครื่องแม่ข่าย

วัชรปกรณ มีชัย

สุวัชรชาติ อาษาสร้อย

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต

สาขาวิชา วิศวกรรมคอมพิวเตอร์

คณะอุตสาหกรรมและเทคโนโลยี มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน

วิทยาเขตสกลนคร พ.ศ. 2569

INTELLIGENT SERVER OPERATIONS PLATFORM

ระบบปฏิบัติการและบริหารจัดการเครื่องแม่ข่าย

วัชรปกรณ มีชัย

สุวัชรชาติ อาษาสร้อย

อาจารย์ที่ปรึกษา: อาจารย์อัษฎาธร เพชรพรรณ

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรวิศวกรรมศาสตรบัณฑิต

สาขาวิชา วิศวกรรมคอมพิวเตอร์

ปีที่ปริญญานิพนธ์สำเร็จ พ.ศ. 2569

บทคัดย่อ

ปริญญานิพนธ์นี้พัฒนาโปรแกรมสำหรับช่วยผู้ดูแลระบบติดตามทรัพยากรของเครื่องคอมพิวเตอร์และจัดการ Docker container จากจุดเดียว โปรแกรมอ่านการใช้หน่วยประมวลผล หน่วยความจำ และพื้นที่จัดเก็บของเครื่องที่เปิดโปรแกรม พร้อมบันทึกค่าลงไฟล์ CSV เพื่อดูย้อนหลัง ส่วน container สามารถนำเข้ารายการบริการจากไฟล์ Docker Compose แล้วตรวจสอบสถานะ เรียกดู log สั่งเริ่ม หยุด หรือเริ่มใหม่ และกำหนดเพดาน CPU กับ RAM ได้

การตรวจสอบสถานะใช้ค่าเฉลี่ยเคลื่อนที่แบบเอ็กซ์โพเนนเชียล (EMA) กับเกณฑ์เข้าและออกจากสถานะผิดปกติที่ต่างกัน ระบบไม่ต้องฝึกโมเดลจากชุดข้อมูล การแจ้งเตือนและการดำเนินการแก้ไขบางอย่างทำงานตามเงื่อนไขที่กำหนด ผู้ใช้ยังเข้าถึงข้อมูลและจัดการ container ผ่านเว็บแดชบอร์ดที่มีหน้าเข้าสู่ระบบได้ และส่งลิงก์แดชบอร์ดทางอีเมลได้เมื่อกำหนดค่า SMTP ส่วนการสำรองข้อมูลสร้างไฟล์ ZIP จากโฟลเดอร์แหล่งข้อมูลที่โปรแกรมกำหนดหรือผู้ใช้นำเข้า ระบบมีตัวเลือกปรับเพดาน RAM ของ container อัตโนมัติ ซึ่งผู้ใช้ต้องเปิดใช้เป็นราย container

หลักฐานการทดสอบที่มีประกอบด้วยรายงานทดสอบเชิงหน้าที่ 10 รายการซึ่งส่งมาพร้อมโครงการ และผล self-test ของไฟล์ EXE บน Windows จำนวน 7 รายการเมื่อวันที่ 23 กันยายน 2569 รายงานระบุว่าผ่านกรณีทดสอบตามรายการ ส่วน self-test ยืนยันการโหลดทรัพยากรและการเปิดโปรแกรมแบบ one-file ได้ ผลดังกล่าวยังไม่ใช่การวัดประสิทธิภาพระยะยาวหรือการทดสอบบนเครื่องแม่ข่ายหลายสภาพแวดล้อม และยังไม่มีข้อมูลสำหรับคำนวณเวลาตรวจจับหรือการลด downtime

คำสำคัญ: เครื่องแม่ข่าย, Docker, การติดตามทรัพยากร, การตรวจจับความผิดปกติ, การบริหารระบบ

Abstract

This project developed an application to help administrators monitor a computer host and manage Docker containers from one interface. It collects CPU, memory, and disk usage from the computer running the application and stores metric samples in CSV files. Docker Compose files can be imported to identify services and containers for monitoring. The application supports container status checks, log viewing, start, stop, restart, and CPU and memory limit updates.

Resource conditions are evaluated using an exponential moving average (EMA) and separate thresholds for entering and leaving an abnormal state. The method is rule-based and does not require a trained machine-learning model. The application also provides in-app alerts, selected automated remediation actions, an authenticated web dashboard, SMTP email delivery for dashboard links, and ZIP backups of configured project folders. An optional per-container RAM limit manager adjusts limits only when enabled by the user.

Available test evidence consists of a submitted report listing ten functional test cases as passed and a Windows executable self-test that passed seven runtime and packaging checks on 23 September 2026. These results cover the reported cases and executable startup checks; they do not establish long-term stability, performance across server environments, detection latency, or reduced downtime.

Keywords: server monitoring, Docker, resource monitoring, anomaly detection, system administration

กิตติกรรมประกาศ

คณะผู้จัดทำขอขอบพระคุณอาจารย์อัษฎาวุธ เพชรพรรณ อาจารย์ที่ปรึกษาปริญญานิพนธ์
ที่ให้คำแนะนำและข้อเสนอแนะระหว่างการดำเนินงาน

ขอขอบพระคุณคณาจารย์และบุคลากรของสาขาวิชาและคณะอุตสาหกรรมและเทคโนโลยี

มหาวิทยาลัยเทคโนโลยีราชมงคลอีสาน วิทยาเขตสกลนคร

ที่สนับสนุนการศึกษาและการจัดทำโครงการ รวมถึงครอบครัวและเพื่อนที่ให้การกำลังใจตลอดมา

คณะผู้จัดทำขอมอบคุณค่าและประโยชน์จากปริญญานิพนธ์ฉบับนี้แก่ผู้ที่มีส่วนช่วยเหลือทุกที่
าน หากมีข้อผิดพลาดประการใด คณะผู้จัดทำขอน้อมรับคำแนะนำเพื่อนำไปปรับปรุงต่อไป

คณะผู้จัดทำ

สารบัญ

บทคัดย่อ.....	3
Abstract	4
กิตติกรรมประกาศ	5
สารบัญ	6
สารบัญรูป.....	10
สารบัญตาราง	11
คำอธิบายสัญลักษณ์และคำย่อ	12
บทที่ 1 บทนำ.....	13
1.1 ความเป็นมาและความสำคัญ.....	13
1.2 ลักษณะปัญหาและความต้องการ	15
1.3 วัตถุประสงค์.....	16
1.4 ขอบเขต	16
1.5 ประโยชน์ที่คาดว่าจะได้รับ	18
1.6 คำศัพท์ที่ใช้ในรายงาน.....	18
1.7 คำถามและตัวชี้วัดสำหรับการประเมิน	19
บทที่ 2 แนวคิดและเทคโนโลยีที่เกี่ยวข้อง.....	20
2.1 การติดตามทรัพยากรระบบ.....	20
2.2 การปรับค่าให้เรียบด้วย EMA.....	21
2.3 Hysteresis และเกณฑ์สถานะ	22

2.4 การตรวจจับด้วยกฎและความหมายของ risk score	23
2.5 การควบคุมทรัพยากร container	24
2.6 Docker Compose และ YAML	25
2.7 เว็บแอปพลิเคชันด้วย Flask	Error! Bookmark not defined.
2.8 การเข้าถึงผ่าน Cloudflare Tunnel	25
2.9 CSV และการสำรองข้อมูล	Error! Bookmark not defined.
2.10 ความปลอดภัยและขอบเขตการใช้งาน	Error! Bookmark not defined.
2.11 เปรียบเทียบเครื่องมือและขอบเขตของโครงการ	Error! Bookmark not defined.
2.12 โครงสร้างพื้นฐานเครื่องแม่ข่าย	Error! Bookmark not defined.
2.13 การบริหารงานปฏิบัติการ IT	Error! Bookmark not defined.
2.14 AIOps ในฐานะแนวคิดอ้างอิง	Error! Bookmark not defined.
2.15 แนวทางตรวจจับความผิดปกติที่เกี่ยวข้อง	Error! Bookmark not defined.
2.16 การวิเคราะห์อนุกรมเวลาและการบำรุงรักษาเชิงพยากรณ์	Error! Bookmark not defined.
2.17 มุมมองการเฝ้าระวังบริการ	Error! Bookmark not defined.
2.18 ตัวชี้วัดสำหรับการประเมินการตรวจจับ	Error! Bookmark not defined.
2.19 ขอบเขตการประยุกต์ใช้แนวคิดในระบบ	Error! Bookmark not defined.
2.20 สรุป	Error! Bookmark not defined.
บทที่ 3 การออกแบบและพัฒนา	32
3.1 ภาพรวมการทำงาน	32

3.2 ลำดับการทำงานหลัก	Error! Bookmark not defined.
3.3 รูปแบบข้อมูลและไฟล์	Error! Bookmark not defined.
3.4 การจัดการสถานะและข้อผิดพลาด	Error! Bookmark not defined.
3.5 การเก็บและประเมิน metrics	Error! Bookmark not defined.
3.6 การจัดการ Docker	Error! Bookmark not defined.
3.7 เว็บแดชบอร์ดและการเชื่อมต่อ	Error! Bookmark not defined.
3.8 การแจ้งเตือนและการดำเนินการอัตโนมัติ	Error! Bookmark not defined.
3.9 การสำรองข้อมูล	Error! Bookmark not defined.
3.10 เครื่องมือและสภาพแวดล้อมที่ยืนยันได้	Error! Bookmark not defined.
3.11 การสร้างไฟล์ส่งมอบ	Error! Bookmark not defined.
3.12 วิธีประเมินผล	Error! Bookmark not defined.
บทที่ 4 ผลการพัฒนาและการทดสอบ	42
4.1 ผลการพัฒนา	42
4.2 หน้าจอของโปรแกรม	43
4.3 รายงานผลทดสอบเชิงหน้าที่	43
4.4 ผล self-test ของ Windows EXE	45
4.5 อภิปรายผล	47
4.6 ข้อจำกัดของผลทดสอบ	Error! Bookmark not defined.
บทที่ 5 สรุปผลและข้อเสนอแนะ	49
5.1 สรุปผล	49
5.2 ข้อจำกัดของระบบ	49

5.3 ข้อเสนอแนะสำหรับการพัฒนาต่อ.....	50
5.4 งานที่ควรทำก่อนส่งฉบับสมบูรณ์	50
บรรณานุกรม	51
ภาคผนวก ก	53
แนวทางการใช้งาน.....	54
1. เตรียมเครื่อง.....	54
2. เปิดโปรแกรม	54
3. เพิ่มบริการ.....	54
4. จัดการ container	54
5. ใช้เว็บแดชบอร์ด.....	54
6. สำรองข้อมูล.....	55
ภาคผนวก ข.....	56
ภาคผนวก ค	57
ประวัติผู้เขียน	58

สารบัญรูป

รูปที่ 4.1 หน้าหลักสำหรับติดตามทรัพยากร

รูปที่ 4.2 หน้าประวัติและกราฟการใช้ทรัพยากร

รูปที่ 4.3 หน้า Server Dashboard

รูปที่ 4.4 หน้า Server Manager

รูปที่ 4.5 หน้าต่าง Terminal

รูปที่ 4.6 หน้า Action Log

สารบัญตาราง

ตารางที่ 1.1 ความต้องการของระบบ

ตารางที่ 2.1 เปรียบเทียบขอบเขตเครื่องมือ

ตารางที่ 3.1 หน้าที่ของโมดูล

ตารางที่ 3.2 ลำดับการทำงานหลัก

ตารางที่ 3.3 รูปแบบไฟล์ข้อมูล

ตารางที่ 4.1 รายการทดสอบเชิงหน้าที่

ตารางที่ 4.2 ผลทดสอบที่รายงาน

คำอธิบายสัญลักษณ์และคำย่อ

API	Application Programming Interface
CPU	Central Processing Unit
CSV	Comma-Separated Values
EMA	Exponential Moving Average
HTTP/HTTPS	Hypertext Transfer Protocol / Secure
JSON	JavaScript Object Notation
RAM	Random Access Memory
SMTP	Simple Mail Transfer Protocol
UI	User Interface
YAML	รูปแบบข้อมูลสำหรับการกำหนดค่า; Docker Compose
ใช้ประกาศบริการ	
Host	เครื่องคอมพิวเตอร์ที่เปิดโปรแกรม
Risk score	ค่าสูงสุดของ metrics หลังปรับ EMA ไม่ใช่ความน่าจะเป็น

บทที่ 1

บทนำ

ภาพรวมของการศึกษา โดยเริ่มจากที่มาและความสำคัญของปัญหา เพื่ออธิบายบริบทและเหตุผลที่ทำให้เกิดความสนใจในประเด็นดังกล่าว จากนั้นจะกล่าวถึงวัตถุประสงค์ของการศึกษา ขอบเขตและแนวทางการดำเนินงาน ตลอดจนประโยชน์ที่คาดว่าจะได้รับ เนื้อหาในบทนี้จะช่วยให้ผู้อ่านเข้าใจจุดมุ่งหมายและกรอบของการศึกษาอย่างชัดเจน รวมถึงเป็นพื้นฐานสำหรับทำความเข้าใจรายละเอียดในบทถัดไป ซึ่งจะกล่าวถึงแนวคิด ทฤษฎี วิธีดำเนินงาน และผลการศึกษาตามลำดับ

1.1 ความเป็นมาและความสำคัญ

1.2 ลักษณะปัญหาและความต้องการ

1.3 วัตถุประสงค์

1.4 ขอบเขต

1.5 ระยะเวลาและแผนการดำเนินโครงการ

1.6 ประโยชน์ที่คาดว่าจะได้รับ

1.7 คำศัพท์ที่ใช้ในรายงาน

1.8 คำถามและตัวชี้วัดสำหรับการประเมิน

1.1 ความเป็นมาและความสำคัญ

ระบบสารสนเทศของสถาบันการศึกษาและองค์กรทั่วไปอาศัยเครื่องมือช่วยรองรับงานหลายประเภท เช่น เว็บไซต์ ระบบข้อมูล และบริการภายใน

ผู้ดูแลจึงต้องติดตามทั้งภาระทรัพยากรและสถานะบริการ หากมีหลายบริการหรือใช้ container การตรวจทีละหน้าจอและสลับคำสั่งไปมาอาจทำให้มองภาพรวมได้ยากขึ้น

ข้อความเป็นบริบทของปัญหา

ไม่ได้หมายความว่าโครงการถูกติดตั้งในศูนย์ข้อมูลของมหาวิทยาลัยหรือมีข้อมูลใช้งานจริงจากระบบทะเบียน/ระบบสอบ

โครงการนี้เริ่มจากความต้องการรวมงานพื้นฐานดังกล่าวไว้ในโปรแกรมเดียว

ผู้ใช้ดูแลทรัพยากรของเครื่อง จัดการ container ที่เลือกจากไฟล์ Docker Compose และเปิดหน้าเว็บเพื่อดูสถานะหรือสั่งงานบางอย่างได้ การแจ้งเตือนอาศัยเกณฑ์ที่กำหนดไว้ล่วงหน้า

จึงตอบสนองได้โดยไม่ต้องเตรียมข้อมูลฝึกโมเดล แต่ยังคงเลือกเกณฑ์ให้เหมาะกับลักษณะงานจริง

ชื่อโครงการใช้คำว่า Intelligent Server Operations Platform ตามชื่อระบบที่พัฒนา

กลไกตรวจจับในรุ่นที่ตรวจสอบเป็นการคำนวณ EMA และกฎ threshold ไม่ใช่โมเดล machine learning ที่ฝึกจากข้อมูล รายงานฉบับนี้จึงอธิบายวิธีทำงานตามโค้ดและหลักฐานที่มี

และไม่ถือว่าระบบหาสาเหตุของปัญหาหรือรับประกันการป้องกัน downtime

นอกจากนี้ ระบบยังรวบรวมการบริหารจัดการ Docker Container และการใช้สถานะบริการผ่าน Dashboard ศูนย์กลาง เพื่อลดความซับซ้อนในการสลับหน้าจอทำงานของผู้ดูแลระบบ

1.2 ลักษณะปัญหาและความต้องการ

ปัญหาที่โครงการเลือกแก้เป็นงานติดตามพื้นฐานที่ผู้ดูแลต้องทำซ้ำ เช่น ตรวจสอบทรัพยากร เช็กสถานะ container และดู log หากแยกใช้หลายเครื่องมือ ผู้ใช้ต้องเสียเวลาสลับหน้าจอ โครงการจึงรวบรวมข้อมูลเบื้องต้นทั้งหมดไว้ในแอปเดียว

ผู้ดูแลระบบที่มีสิทธิ์เข้าถึงเครื่องและ Docker โดยระบบจะไม่ทำหน้าที่แทนระบบเฝ้าระวังขององค์กร และจัดการเครื่องปลายทางผ่าน Docker หรือวิธีที่รองรับเท่านั้น ทั้งนี้จะติดตามเฉพาะค่าทรัพยากร เฉพาะรายการที่ผู้ใช้เลือกนำเข้า

รหัส	ความต้องการ	พฤติกรรมที่คาดหวัง
F01	ดูทรัพยากรเครื่อง	แสดง CPU, RAM และ disk ของเครื่องที่แอปกำลังทำงาน
F02	ดูประวัติ	อ่านค่าล่าสุดจาก CSV และนำไปแสดงกราฟ
F03	ตรวจสอบสถานะ	ปรับค่า EMA และเปลี่ยนสถานะเมื่อเข้า/ออก threshold
F04	จัดการ Docker	แสดงรายการที่เลือก และรองรับ start/stop/restart/logs/exec
F05	กำหนด resource limit	บันทึก CPU/RAM limit และส่งคำสั่งปรับให้ Docker
F06	ดูผ่านเว็บ	ต้อง login ก่อนเข้าหน้า dashboard และ API สำหรับจัดการ
F07	สำรองข้อมูล	สร้าง ZIP ตามรายการแหล่งข้อมูลและปลายทางที่ตั้งไว้
F08	ปรับ RAM อัตโนมัติ	ทำงานเฉพาะ container ที่ผู้ใช้เปิดตัวเลือกไว้

ข้อกำหนดด้านการใช้งานคือหน้าจอกควรเข้าถึงข้อมูลหลักได้โดยไม่ต้องจำคำสั่ง Docker สำหรับงานทั่วไป และข้อมูลการตั้งค่าต้องยังอยู่หลังปิดโปรแกรม ส่วนข้อกำหนดด้านความปลอดภัยและเสถียรภาพยังต้องประเมินเพิ่มเติม โดยเฉพาะสิทธิ์ Docker, การเปิด tunnel, การเก็บรหัสผ่าน SMTP และการยืนยันผลการกู้คืน backup

1.3 วัตถุประสงค์

- พัฒนาโปรแกรมเดสก์ท็อปสำหรับติดตาม CPU หน่วยความจำ และพื้นที่จัดเก็บของเครื่องที่ใช้งาน
- พัฒนากลไกประเมินสถานะทรัพยากรด้วย EMA และเกณฑ์เข้าออกสถานะที่กำหนด
- รวมการจัดการ Docker container และการกำหนดเพดาน CPU/RAM ไว้ในส่วนติดต่อเดียว พร้อมตัวเลือกปรับ RAM อัตโนมัติ
- จัดทำกลไกสำรองไฟล์โครงการที่เลือกในรูปแบบ ZIP

1.4 ขอบเขต

- พัฒนาโปรแกรมด้วย Python และส่วนติดต่อเดสก์ท็อป CustomTkinter โดยไฟล์ส่งมอบที่บันทึกไว้เป็น Windows EXE แบบ one-file
- อ่านค่าทรัพยากรของเครื่องที่โปรแกรมทำงานอยู่ด้วย psutil; พื้นที่ดิสก์อ่านจากพาร์ทิชันระบบหลัก (C:\ บน Windows หรือ / บน Linux)
- ใช้ Docker CLI เพื่ออ่านสถานะ สั่งเริ่ม/หยุด/เริ่มใหม่ ดู log และส่งคำสั่งเข้า container ที่เลือก
- นำเข้า YAML ที่เป็น Docker Compose เพื่อระบุบริการและ container; ไฟล์ Compose ของแอปเองใช้เปิดบริการเว็บและ tunnel เมื่อกำหนดไว้
- ให้เว็บแดชบอร์ดรับการเข้าสู่ระบบ และป้องกันเส้นทาง API สำหรับการจัดการด้วยการตรวจสอบ session
- รองรับการส่งลิงก์ dashboard ผ่าน SMTP เมื่อผู้ใช้กำหนดอีเมลผู้ส่งและผู้รับ
- มีระบบปรับ RAM ของ container ที่ผู้ใช้เปิดสวิตช์ราย container; ใช้ EMA, headroom และเพดานส่วนกลางในการคำนวณ
- สำรองไฟล์/โฟลเดอร์ที่ระบบเลือกเป็น ZIP; ไม่สำรอง host ทั้งเครื่องหรือ Docker volumes โดยอัตโนมัติ

1.6 ประโยชน์ที่คาดว่าจะได้รับ

ประโยชน์ที่คาดว่าจะได้รับจากโครงการนี้คือการช่วยให้ผู้ดูแลระบบมีศูนย์กลางสำหรับติดตามทรัพยากร และบริหารจัดการ container ได้จากหน้าจอเดียว ซึ่งช่วยลดความยุ่งยากจากการสลับคำสั่งหรือสลับหน้าจอไปมา การรวบรวมข้อมูลสถานะที่จำเป็นและการดูประวัติย้อนหลังในรูปแบบไฟล์ CSV ช่วยให้การตรวจสอบแนวโน้ม ของระบบทำได้สะดวกยิ่งขึ้น

นอกจากนี้ การนำเกณฑ์การตรวจวัดพร้อมกลไกการตอบสนองอัตโนมัติมารวมไว้ในแอปเดียว ยังเป็นตัวอย่างแนวคิดและเป็นโครงสร้างพื้นฐานในการพัฒนาเครื่องมือดูแลระบบขนาดเล็กที่มีประสิทธิภาพ ซึ่งช่วยลดภาระงานประจำวันและลดโอกาสเกิดความผิดพลาดในการพิมพ์คำสั่งของผู้ดูแลระบบได้อย่างมีนัยสำคัญ ทั้งนี้ ระบบยังช่วยเพิ่มความคล่องตัวในการตรวจสอบและจัดการบริการเบื้องต้นได้อย่างรวดเร็วและทันท่วงที อย่างไรก็ดี ประโยชน์ในเชิงปริมาณ เช่น การลดเวลาในการทำงานอย่างชัดเจน หรือความสามารถในการป้องกัน เกิด downtime ของระบบได้อย่างเต็มประสิทธิภาพนั้น ยังคงต้องมีการประเมินและเก็บผลวัดค่า

1.7 คำศัพท์ที่ใช้ในรายงาน

คำ	ความหมาย
Container	หน่วยสำหรับรันแอปพลิเคชันที่จัดการด้วย Docker
Docker Compose	รูปแบบไฟล์ YAML สำหรับประกาศบริการและค่าที่เกี่ยวข้องกับ Docker Compose
EMA	ค่าเฉลี่ยที่ให้น้ำหนักข้อมูลล่าสุดมากกว่าข้อมูลก่อนหน้า
Hysteresis	การใช้เกณฑ์คนละค่าตอนเข้าสถานะและออกจากสถานะเพื่อลดการเปลี่ยนสถานะถี่
Remediation	การดำเนินการแก้ไขตามเงื่อนไขที่โปรแกรมกำหนด
One-file executable	ไฟล์ EXE ที่บรรจุทรัพยากรที่จำเป็นเพื่อเปิดโปรแกรมจากไฟล์เดียว

1.8 คำถามและตัวชี้วัดสำหรับการประเมิน

การประเมินระบบควรแยกคำถามเชิงหน้าที่ออกจากคำถามเชิงประสิทธิภาพ

คำถามเชิงหน้าที่ตรวจว่าโปรแกรมอ่าน metric ได้ เปลี่ยนสถานะตามเกณฑ์ สั่งงาน Docker และเปิดเว็บได้หรือไม่

ส่วนคำถามเชิงประสิทธิภาพต้องใช้ข้อมูลหลายรอบและสภาพแวดล้อมที่ระบุชุด เช่น

เวลาจากเริ่มเหตุถึงแจ้งเตือน อัตราการตรวจพบ และจำนวน false alarm

คำถาม	ตัวชี้วัดที่เหมาะสม	สถานะหลักฐาน
ค่าที่แสดงอ่านได้หรือไม่	เปรียบเทียบกับเครื่องมือ OS พร้อม timestamp	มีรายงานว่ายืนยันแล้ว แต่ไม่มีตัวเลขแนบ
สถานะ anomaly เปลี่ยนตามกฎหรือไม่	trace ของค่าเข้า/EMA/สถานะ	รายงานเดิมระบุผ่าน ไม่มี trace
ระบบตรวจพบเหตุได้ครบเพียงใด	$\text{Recall} = \text{TP}/(\text{TP}+\text{FN})$	ยังไม่มี labelled event set
การแจ้งเตือนเกินจริงหรือไม่	$\text{Precision} = \text{TP}/(\text{TP}+\text{FP})$	ยังไม่มีผลคำนวณ
การกู้คืนเร็วขึ้นหรือไม่	MTTR และ baseline ก่อน/หลัง	ไม่มีข้อมูลเปรียบเทียบ
โปรแกรมเสถียรแค่ไหน	uptime, error count, resource overhead	ยังไม่มี long-run report

ตัวชี้วัดในตารางเป็นกรอบสำหรับเก็บหลักฐานในอนาคต ไม่ใช่ผลของโครงการในปัจจุบัน

การตั้งเป้าหมายเชิงตัวเลข เช่น ลด MTTR หรือกำหนด availability ต้องระบุวิธีวัด ช่วงเวลา และ baseline ล่วงหน้า และยังไม่สรุปไม่ได้หากไม่มีข้อมูลทดลองรองรับ

บทที่ 2

แนวคิดและเทคโนโลยีที่เกี่ยวข้อง

บทนี้กล่าวถึงแนวคิด หลักการ และเทคโนโลยีที่เกี่ยวข้องกับการศึกษา โดยรวบรวมข้อมูลพื้นฐานที่จำเป็นต่อการทำความเข้าใจเนื้อหาและแนวทางการดำเนินงาน เริ่มจากการอธิบายแนวคิด และทฤษฎีสำคัญที่ใช้เป็นกรอบในการศึกษา จากนั้นนำเสนอเทคโนโลยี เครื่องมือ และองค์ประกอบต่าง ๆ ที่เกี่ยวข้อง พร้อมอธิบายบทบาทและความสำคัญของแต่ละส่วน เนื้อหาในบทนี้จะช่วยให้เห็นความเชื่อมโยงระหว่างพื้นฐานทางทฤษฎีกับการนำไปประยุกต์ใช้ และเป็นแนวทางสำหรับการออกแบบและพัฒนางานในบทถัดไป

- 2.1 แนวคิดและทฤษฎีเกี่ยวกับระบบ
- 2.2 การวิเคราะห์และการออกแบบระบบ
- 2.3 การติดตามและการประมวลผลทรัพยากรระบบ
- 2.4 การตรวจจับความผิดปกติและการประเมินความเสี่ยง
- 2.5 การวิเคราะห์อนุกรมเวลาและการบำรุงรักษาเชิงพยากรณ์
- 2.6 การเฝ้าระวังและการบริหารบริการ
- 2.7 เทคโนโลยี Container และการจัดการระบบ
- 2.8 การพัฒนาเว็บแอปพลิเคชันและการเข้าถึงระบบ
- 2.9 การจัดเก็บและสำรองข้อมูล
- 2.10 เครื่องมือที่ใช้ในการพัฒนา
- 2.11 AIOps และแนวคิดที่เกี่ยวข้อง
- 2.12 ความปลอดภัยและขอบเขตการใช้งานระบบ
- 2.13 การเปรียบเทียบและขอบเขตของโครงการ
- 2.15 แนวทางตรวจจับความผิดปกติที่เกี่ยวข้อง
- 2.14 สรุป

2.1 แนวคิดและทฤษฎีเกี่ยวกับระบบ

โครงสร้างพื้นฐานของระบบสารสนเทศสามารถพิจารณาได้เป็นลำดับชั้น ตั้งแต่ฮาร์ดแวร์ ระบบปฏิบัติการ ระบบ runtime หรือ virtualization เครือข่าย ไปจนถึงแอปพลิเคชันและบริการที่ผู้ใช้เรียกใช้งาน แต่ละชั้นสามารถเป็นจุดเริ่มต้นของปัญหาที่ส่งผลต่อการทำงานของระบบได้ เช่น พื้นที่จัดเก็บข้อมูลไม่เพียงพอในชั้นจัดเก็บข้อมูล หรือ process ใช้หน่วยความจำสูงในชั้นแอปพลิเคชัน

การบริหารระบบจึงจำเป็นต้องมีการติดตามสถานะของทรัพยากรและบริการที่เกี่ยวข้อง เพื่อให้ผู้ดูแลระบบสามารถตรวจสอบภาระงานและระบุสถานะที่ควรตรวจสอบเพิ่มเติมได้ โครงการนี้ เน้นการติดตามทรัพยากรในระดับเครื่องแม่ข่ายและ container โดยข้อมูลที่ได้เป็นเพียงส่วนหนึ่งของ ภาพรวมโครงสร้างพื้นฐาน ไม่ได้ครอบคลุมทุกชั้นของระบบ เช่น network traffic, latency, hardware sensor หรือระบบ cluster

ในด้านการปฏิบัติงาน IT การติดตามสถานะเป็นส่วนหนึ่งของกระบวนการตรวจสอบและตอบสนองต่อเหตุการณ์ โดยข้อมูล monitoring สามารถช่วยระบุอาการของปัญหาและนำไปใช้ประกอบการวิเคราะห์และแก้ไขปัญหาได้ อย่างไรก็ตาม ระบบอัตโนมัติไม่สามารถทดแทนการกำหนด ผู้รับผิดชอบ แผน incident response หรือกระบวนการสำรองและกู้คืนข้อมูลได้ทั้งหมด

แนวคิดดังกล่าวทำให้การบริหารระบบต้องพิจารณาข้อมูลจากหลายระดับร่วมกัน เพื่อให้เห็นความสัมพันธ์ระหว่างทรัพยากรของเครื่องแม่ข่ายและการทำงานของบริการที่เกี่ยวข้อง

การติดตามข้อมูลอย่างต่อเนื่องยังช่วยให้สามารถเปรียบเทียบสถานะของระบบในช่วงเวลาต่าง ๆ และใช้เป็นข้อมูลประกอบการตัดสินใจของผู้ดูแลระบบได้

ดังนั้น ระบบบริหารจัดการจึงควรมีการรวบรวมข้อมูลสถานะอย่างเป็นระบบ พร้อมกำหนดขอบเขตของการตรวจสอบและการดำเนินการให้สอดคล้องกับความสามารถของระบบ

2.2 การวิเคราะห์และการออกแบบระบบ

การวิเคราะห์และการออกแบบระบบเป็นกระบวนการกำหนดองค์ประกอบ หน้าที่ และความสัมพันธ์ระหว่างส่วนต่าง ๆ ของระบบ เพื่อให้สามารถพัฒนาระบบให้สอดคล้องกับวัตถุประสงค์และขอบเขตที่กำหนด

สำหรับโครงการนี้ ระบบสามารถแบ่งการทำงานหลักออกเป็นส่วนของการอ่านข้อมูล ทรัพยากร การประมวลผลและประเมินสถานะ การจัดการ container การให้บริการผ่านเว็บ และการจัดเก็บข้อมูลย้อนหลัง โดยการอ่านทรัพยากรของเครื่องใช้ข้อมูลจากระบบปฏิบัติการ ขณะที่ข้อมูลของ container ถูกอ่านผ่าน Docker CL

องค์ประกอบ	หน้าที่
Host Monitoring	อ่านค่า CPU, RAM และ Disk ของเครื่อง
Resource Processing	ประมวลผลและปรับค่า Metric ด้วย EMA
Status Detection	ประเมินสถานะจาก Threshold และ Hysteresis
Risk Score	แสดงค่าความเสี่ยงจากค่าที่ผ่านการปรับเรียบ
Container Management	ตรวจสอบและควบคุม Container ผ่าน Docker
Web Application	แสดง Dashboard และให้บริการ API
Data Storage	จัดเก็บข้อมูล Metric ในรูปแบบ CSV
Backup	สำรองข้อมูลตามช่วงเวลาที่กำหนด
External Access	เข้าถึงเว็บจากภายนอกผ่าน Cloudflare Tunnel

ตารางที่ 2.1 องค์ประกอบหลักของระบบ

2.3 การติดตามและการประมวลผลทรัพยากรระบบ

การติดตามทรัพยากรระบบช่วยให้สามารถตรวจสอบภาระงานของเครื่องในช่วงเวลาต่าง ๆ ได้ โครงการนี้ใช้ไลบรารี psutil ในภาษา Python เพื่ออ่านค่าทรัพยากรของระบบ ได้แก่ CPU, RAM และพื้นที่จัดเก็บข้อมูล โดยค่าที่อ่านได้เป็นค่าของเครื่องที่โปรแกรมกำลังทำงานอยู่ ส่วนข้อมูลของ container ใช้ Docker CLI แยกต่างหาก

เนื่องจากค่าการใช้ทรัพยากรสามารถเปลี่ยนแปลงอย่างรวดเร็ว ระบบจึงใช้ Exponential Moving Average (EMA) เพื่อปรับค่าให้เรียบก่อนนำไปใช้ในการตัดสินใจ โดยสมการที่ใช้คือ

$$E_t = \alpha X_t + (1 - \alpha)E_{t-1}$$

โดยที่ X_t คือค่าที่อ่านได้ในรอบปัจจุบัน E_{t-1} คือค่า EMA ในรอบก่อนหน้า และ α คือ ค่าน้ำหนักของข้อมูลใหม่ ซึ่งโครงการกำหนด $\alpha = 0.3$

ค่า EMA ที่ได้จะถูกนำมาใช้ร่วมกับกลไก Hysteresis เพื่อป้องกันการเปลี่ยนสถานะไปมา เมื่อค่าทรัพยากรอยู่ใกล้เกณฑ์ โดยระบบกำหนดให้เข้าสู่สถานะผิดปกติเมื่อค่า EMA สูงสุดของ CPU, RAM หรือ Disk มากกว่า 90% และกลับสู่สถานะปกติเมื่อค่าดังกล่าวต่ำกว่า 80% ส่วนช่วงระหว่าง 80–90% จะคงสถานะเดิมไว้

ค่า EMA	สถานะ
มากกว่า 90%	เปลี่ยนเป็นผิดปกติ
80–90%	คงสถานะเดิม
ต่ำกว่า 80%	กลับเป็นปกติ

ตารางที่ 2.2 เกณฑ์การเปลี่ยนสถานะ

2.4 การตรวจจับความผิดปกติและการประเมินความเสี่ยง

ระบบใช้กฎและค่าขีดจำกัดที่กำหนดไว้ล่วงหน้าในการตรวจจับสถานะที่ควรให้ผู้ดูแลระบบตรวจสอบเพิ่มเติม โดยค่าที่เกินเกณฑ์เป็นเพียงสัญญาณสำหรับการตรวจสอบ ไม่ใช่การวินิจฉัยสาเหตุของปัญหา และไม่จำเป็นต้องมีการฝึกโมเดล สำหรับ risk score ระบบนำค่า EMA ของ CPU, RAM และ Disk มาหาค่าสูงสุด โดยสามารถเขียนได้เป็น

$$Risk = \max(E_{CPU}, E_{RAM}, E_{Disk})$$

ดังนั้น risk score จึงเป็นค่าตัวชี้วัดที่ผ่านการปรับเรียบและมีค่าสูงสุดในขณะนั้น ไม่ได้หมายถึงความน่าจะเป็นที่ระบบจะล้มเหลว แนวทางการตรวจจับความผิดปกติมีทั้ง Supervised, Unsupervised และ Semi-supervised โดย Isolation Forest เป็นตัวอย่างของวิธี Unsupervised อย่างไรก็ตาม โครงการรุ่นปัจจุบันไม่ได้ import หรือฝึก Isolation Forest จึงควรนำเสนอเป็นแนวคิดที่เกี่ยวข้องหรือแนวทางต่อยอด ไม่ใช่ความสามารถที่มีอยู่ในระบบ

แนวทาง	ลักษณะ
Supervised	ใช้ข้อมูลที่มีป้ายกำกับปกติ/ผิดปกติ
Unsupervised	ค้นหารูปแบบหรือข้อมูลที่แตกต่างจากข้อมูลส่วนใหญ่โดยไม่ใช้ Label
Semi-supervised	สร้างแบบจำลองจากข้อมูลปกติแล้วนำไปตรวจข้อมูลใหม่
Isolation Forest	ตัวอย่างวิธี Unsupervised สำหรับแยกข้อมูลที่มีลักษณะผิดปกติ

ตารางที่ 2.3 แนวทางการตรวจจับความผิดปกติ

สำหรับการประเมินประสิทธิภาพการตรวจจับ หากมีชุดข้อมูลที่ติดป้ายกำกับ สามารถใช้ True Positive, False Positive, True Negative และ False Negative เพื่อคำนวณ Precision, Recall และ F1-score ได้ โดยไม่ควรใช้ Accuracy เพียงตัวเดียวในกรณีที่ข้อมูลผิดปกติมีจำนวนน้อย

2.5 การวิเคราะห์อนุกรมเวลาและการบำรุงรักษาเชิงพยากรณ์

ข้อมูล Metric ที่บันทึกพร้อมเวลาเป็นข้อมูลอนุกรมเวลา การใช้ EMA หรือ Exponential Smoothing ช่วยลดความผันผวนของข้อมูลโดยให้น้ำหนักกับข้อมูลล่าสุดมากกว่าข้อมูลในอดีต อย่างไรก็ตาม การปรับค่าให้เรียบไม่เท่ากับการพยากรณ์ระยะยาว ซึ่งควรมีการแบ่งข้อมูลทดสอบและประเมินค่าความคลาดเคลื่อน

โครงการมีการแสดงแนวโน้ม CPU จากการเปรียบเทียบค่าเฉลี่ยของสองช่วงและการประมาณแนวโน้มแบบเส้นตรงใน brain.py แต่ยังไม่มียุทธศาสตร์สำหรับประเมิน forecast error ดังนั้นควรเรียกว่าแนวโน้มหรือค่าประมาณ และไม่ควรสรุปเป็นค่าความแม่นยำหรือวันที่ระบบจะเกิดเหตุการณ์ในอนาคต

2.6 การเฝ้าระวังและการบริหารบริการ

การเฝ้าระวังระบบสามารถพิจารณา Latency, Traffic, Errors และ Saturation ร่วมกับการใช้ทรัพยากร เพื่อให้เห็นสถานะของบริการได้ครอบคลุมมากขึ้น โดยระบบในโครงการครอบคลุมการใช้ทรัพยากรที่เกี่ยวข้องกับ Saturation และสถานะของ Container แต่ยังไม่ได้เก็บ Latency, Request Traffic หรือ Error Rate โดยตรง

ในด้าน IT Operations ข้อมูล Monitoring สามารถใช้ระบุอาการของปัญหาและช่วยวิเคราะห์แก้ไขปัญหาได้ หลังการแก้ไขควรตรวจสอบผลลัพธ์ซ้ำ พร้อมกำหนดขอบเขตของ Automation, Cooldown และวิธีตรวจสอบผลของคำสั่งให้ชัดเจน

2.7 เทคโนโลยี Container และการจัดการระบบ

Docker เป็นเทคโนโลยีที่ใช้ในการสร้างและจัดการ Container ซึ่งโครงการนำมาใช้ในการตรวจสอบและควบคุมทรัพยากรของ Container โดยสามารถกำหนดข้อจำกัดด้าน CPU และหน่วยความจำได้ ระบบเปิดให้ผู้ใช้กำหนดค่า CPU เป็นเปอร์เซ็นต์และ RAM เป็นเมกะไบต์ โดยใช้คำสั่ง Docker update สำหรับปรับค่า ขณะเดียวกันระบบมีฟังก์ชันปรับ RAM อัตโนมัติที่สามารถเลือกเปิดใช้งานได้ โดยอ่าน RAM ที่ใช้จริงและนำมาคำนวณร่วมกับ EMA, Headroom และ Ceiling Docker Compose ใช้ไฟล์ YAML เพื่ออธิบายบริการหลายรายการและความสัมพันธ์ระหว่างบริการ สำหรับการเปิดใช้งานร่วมกัน โครงการสามารถอ่านไฟล์ YAML ที่ผู้ใช้เลือกเพื่อแสดงบริการและนำ Container ที่เกี่ยวข้องเข้าสู่รายการดูแล โดยไฟล์ docker-compose.yml ที่มากับแอปเป็นอีกส่วนหนึ่งและมีหน้าที่เกี่ยวข้องกับ Dashboard หรือ Tunnel

ทรัพยากร	การทำงาน
CPU	กำหนด CPU เป็นเปอร์เซ็นต์
RAM	กำหนด RAM เป็น MB
CPU 100%	หมายถึงไม่จำกัด Quota ตามการออกแบบของระบบ
Automatic RAM	เลือกเปิดใช้งานได้
EMA	ใช้ช่วยประเมินแนวโน้มการใช้ RAM
Headroom / Ceiling	ใช้ประกอบการคำนวณค่าที่เสนอ
Docker Update	ใช้ส่งคำสั่งปรับทรัพยากร

ตารางที่ 2.4 การจัดการทรัพยากร Container

2.8 การพัฒนาเว็บแอปพลิเคชันและการเข้าถึงระบบ

Flask เป็น Web Framework สำหรับภาษา Python ที่สามารถเชื่อม URL เข้ากับฟังก์ชันของแอปพลิเคชันและสามารถ Render Template ได้ โครงการใช้ Flask ในการให้บริการหน้า Login, Dashboard และ API สำหรับอ่านสถานะหรือส่งคำสั่งไปยัง Docker โดยหน้าเว็บเรียก API เป็นระยะเพื่อปรับปรุงข้อมูลที่แสดงผล

การออกแบบในลักษณะดังกล่าวช่วยให้ผู้ดูแลระบบสามารถตรวจสอบสถานะและดำเนินการจัดการระบบผ่านส่วนติดต่อเว็บได้จากจุดเดียว

สำหรับการเข้าถึงระบบจากภายนอก โครงการสามารถใช้ Cloudflare Quick Tunnel เพื่อสร้าง URL สาธารณะสำหรับการทดลองเข้าถึงเว็บจากเครือข่ายภายนอก โดย URL จะถูกสร้างใหม่เมื่อมีการเชื่อมต่อ และเหมาะสำหรับการทดสอบและพัฒนา ไม่ควรตีความว่าเป็นรูปแบบการให้บริการ Production โดยอัตโนมัติ

2.9 การจัดเก็บและสำรองข้อมูล

โครงการบันทึกค่าทรัพยากรพร้อมเวลาลงในไฟล์ CSV โดยประกอบด้วยข้อมูล CPU, RAM และ Disk ทำให้สามารถเปิดดูข้อมูลย้อนหลังหรือใช้ข้อมูลเพื่อสร้างกราฟได้ การจัดเก็บในรูปแบบดังกล่าวช่วยให้ข้อมูลสามารถนำไปตรวจสอบและวิเคราะห์ย้อนหลังได้สะดวก

ระบบสำรองข้อมูลโดยสร้างไฟล์ ZIP จากแหล่งข้อมูลที่กำหนด สามารถตั้งให้เริ่มทำงานทันทีและทำงานซ้ำทุกหนึ่งชั่วโมงเมื่อเปิดใช้งาน ระบบเก็บไฟล์สำรองชุดล่าสุดและสร้างไฟล์ชั่วคราวก่อนแทนที่ไฟล์เดิม เพื่อลดความเสี่ยงที่ไฟล์เดิมจะสูญหายหากกระบวนการสร้างไฟล์ใหม่ผิดพลาด

อย่างไรก็ตาม กลไกดังกล่าวยังไม่ถือเป็นการทดแทนการสำรอง Volume หรือการทดสอบกระบวนการกู้คืนข้อมูล ดังนั้นการใช้งานจริงควรมีการวางแผนสำรองและตรวจสอบความสามารถในการกู้คืนอย่างสม่ำเสมอ

2.10 เครื่องมือที่ใช้ในการพัฒนา

เครื่องมือที่ใช้ในการพัฒนาระบบสามารถแบ่งตามหน้าที่ได้เป็นเครื่องมือสำหรับอ่านข้อมูลระบบ เครื่องมือสำหรับการจัดการ Container เครื่องมือสำหรับพัฒนาเว็บ และเครื่องมือสำหรับการเข้าถึงระบบจากภายนอก

เครื่องมือ/เทคโนโลยี	หน้าที่ในการพัฒนา
Python	ภาษาหลักในการพัฒนาระบบ
psutil	อ่านข้อมูล CPU, RAM, Disk และข้อมูลระบบ
Flask	พัฒนา Web Application และ API
Docker	จัดการ Container
Docker CLI	อ่านสถานะและส่งคำสั่งไปยัง Container
Docker Compose	จัดการบริการหลายรายการร่วมกัน
YAML	ใช้กำหนดโครงสร้างบริการของ Docker Compose
CSV	จัดเก็บข้อมูล Metric ย้อนหลัง
ZIP	ใช้จัดทำไฟล์สำรองข้อมูล
Cloudflare Quick Tunnel	ใช้สำหรับทดลองเข้าถึง Web Application จากภายนอก

ตารางที่ 2.5 เครื่องมือที่ใช้ในการพัฒนา

2.11 AIOps และแนวคิดที่เกี่ยวข้อง

AIOps เป็นแนวทางที่นำข้อมูลจากการปฏิบัติงานด้าน IT และการวิเคราะห์ข้อมูล รวมถึงในบางกรณี Machine Learning มาใช้สนับสนุนการบริหารจัดการระบบ แนวคิดดังกล่าวครอบคลุมงานหลายประเภท ตั้งแต่การจัดการทรัพยากร การตรวจจับความผิดปกติ ไปจนถึงการจัดการเหตุการณ์และความล้มเหลว

สำหรับโครงการนี้ การใช้คำว่า Intelligent ไม่ได้หมายความว่าระบบมีโมเดล Machine Learning ที่ผ่านการฝึกครบวงจร เนื่องจากระบบรุ่นปัจจุบันใช้ EMA, Rule และ Threshold ที่กำหนดไว้ล่วงหน้า และไม่มีขั้นตอน Train Model หรือ Model Artifact

ดังนั้นแนวคิด AIOps จึงถูกนำมาใช้เป็นกรอบอ้างอิงในการอธิบายแนวทางการนำข้อมูล Monitoring มาใช้สนับสนุนการตัดสินใจและ Automation ขณะที่ระบบปัจจุบันยังไม่ใช้แพลตฟอร์ม AIOps เต็มรูปแบบ

2.12 ความปลอดภัยและขอบเขตการใช้งานระบบ

การเปิด Endpoint ที่สามารถส่งคำสั่งไปยัง Docker ทำให้บัญชีผู้ใช้บนเว็บมีสิทธิ์ในการดำเนินการกับบริการ ดังนั้นจึงจำเป็นต้องพิจารณาความปลอดภัยของบัญชี เครือข่าย และช่องทางการเข้าถึงระบบ

การใช้งานระบบควรใช้รหัสผ่านที่คาดเดาได้ยาก จำกัดเครือข่ายที่สามารถเข้าถึงระบบ และพิจารณาการใช้งาน HTTPS อย่างเหมาะสม สำหรับ Cloudflare Quick Tunnel ควรใช้ในลักษณะการทดลองหรือพัฒนา และหากนำระบบไปใช้งานจริงควรพิจารณารูปแบบ Tunnel ที่เหมาะสม รวมถึงการป้องกันข้อมูลสำคัญ เช่น รหัสผ่าน SMTP, ข้อมูลบัญชี และไฟล์สำรอง ZIP

2.13 การเปรียบเทียบและขอบเขตของโครงการ

โครงการมีเป้าหมายในการรวมความสามารถพื้นฐานสำหรับผู้ดูแลเครื่องที่ใช้งาน Docker ไว้ในระบบเดียว โดยไม่ได้มีวัตถุประสงค์เพื่อทดแทนแพลตฟอร์ม Monitoring ระดับองค์กร และตารางเปรียบเทียบนี้เป็นการเปรียบเทียบขอบเขตหน้าที่ ไม่ใช่ผลการทดสอบประสิทธิภาพของผลิตภัณฑ์แต่ละชนิด

งาน	เครื่องมือทั่วไป	ขอบเขตของโครงการ
ตรวจ Host	เครื่องมือระบบปฏิบัติการ	แสดง CPU/RAM/Disk ในหน้าจอเดียวและบันทึก CSV
จัดการ Container	Docker CLI / Docker Desktop	เรียกคำสั่งหลักจากแอปและเว็บ
ดูข้อมูลย้อนหลัง	เปิดไฟล์/กราฟภายนอก	อ่าน CSV ล่าสุดและแสดงกราฟในโปรแกรม
เข้าถึงนอก LAN	ตั้งค่า Reverse Proxy / Tunnel เอง	เชื่อม Quick Tunnel ตาม Compose ที่เตรียมไว้
แจ้งเตือน	เครื่องมือ Monitoring เฉพาะทาง	Popup และข้อความในโปรแกรมตามกฎหมายที่กำหนด

ตารางที่ 2.6 การเปรียบเทียบเครื่องมือทั่วไปกับขอบเขตของโครงการ

จากขอบเขตดังกล่าว ระบบจึงมุ่งเน้นการรวมงานพื้นฐานสำหรับการดูแลเครื่องที่ใช้ Docker มากกว่าการเป็นแพลตฟอร์ม Monitoring ระดับองค์กร

แนวคิด/ความสามารถ	สถานะในซอร์สปัจจุบัน	การอธิบายในเล่ม
Isolation Forest / LSTM	ไม่พบโมเดลหรือการฝึก	อธิบายเป็นแนวคิดที่เกี่ยวข้อง/ งานต่อยอด
CPU/RAM/Disk และ Docker	มีการอ่าน/ควบคุมตามโมดูล	อธิบายเป็นความสามารถของ ระบบ
Network, Latency, Logs ทุก ระบบ	ไม่พบการเก็บครบตามขอบเขต เดิม	ระบุเป็นข้อจำกัด
Rollback และ MTTR ลดลง	ไม่มีหลักฐานยืนยัน	ไม่สรุปผลและเสนอให้ทดสอบ
EMA/Hysteresis และ Optional RAM Rule	มีใน brain.py / ram_ai_manager.py	อธิบายพารามิเตอร์จริงและ ข้อจำกัด

ตารางที่ 2.7 ขอบเขตความสามารถของระบบตามสถานะในซอร์สปัจจุบัน

ตารางนี้มีความสำคัญ เพราะช่วยแยกความสามารถที่มีอยู่จริงในซอร์สโค้ดออกจากแนวคิดที่ถูกเสนอไว้เป็นขอบเขตหรือแนวทางต่อยอด โดยเฉพาะ Isolation Forest, LSTM, Network/Latency/Logs และผลด้าน MTTR ซึ่งยังไม่มีหลักฐานยืนยันจากระบบปัจจุบัน

2.14 สรุป

จากแนวคิดและเทคโนโลยีที่เกี่ยวข้อง ระบบเชื่อมโยงการอ่านทรัพยากร การประเมินสถานะ ด้วยกฎ การจัดการ Docker การให้บริการผ่านเว็บ และการจัดเก็บข้อมูล โดยใช้ EMA และ Hysteresis เพื่อลดความผันผวน และใช้ Risk Score แสดงค่าตัวชี้วัดสูงสุดจาก CPU, RAM และ Disk

การประเมินระบบควรแยกความสามารถที่มีจริงในซอร์สโค้ด ผลการทดสอบแพ็คเกจ และผลจากการใช้งานจริง โดย Isolation Forest, LSTM, Network/Latency/Logs และการลด MTTR ยังไม่ควรนำเสนอเป็นความสามารถของระบบปัจจุบัน

บทที่ 3

การออกแบบและพัฒนา

บทนี้นำเสนอขั้นตอนการดำเนินโครงการ การวิเคราะห์และออกแบบระบบ ตลอดจนหลักการการทำงานของโปรแกรม โครงสร้างข้อมูล การจัดการทรัพยากรและ Container การให้บริการผ่านเว็บ และการออกแบบส่วนติดต่อผู้ใช้ โดยมีจุดประสงค์เพื่ออธิบายกระบวนการพัฒนาระบบตั้งแต่การศึกษาข้อมูล การออกแบบองค์ประกอบของระบบ การพัฒนาโปรแกรม ไปจนถึงการทดสอบและประเมินผลการทำงาน

3.1 ลำดับขั้นตอนการดำเนินโครงการ

3.2 การศึกษาและรวบรวมข้อมูล

3.2.1 แนวคิดและเทคโนโลยีที่ศึกษา

3.2.2 การวิเคราะห์ความต้องการและขอบเขตระบบ

3.3 การวิเคราะห์และออกแบบระบบ

3.3.1 ฝั่งการทำงานของระบบ

3.3.2 Context Diagram

3.3.3 ER Diagram

3.3.4 Data Dictionary

3.4 หลักการทำงานของโปรแกรม

3.5 การจัดการข้อมูลและการจัดเก็บ Metrics

3.6 การจัดการ Docker และการทำงานอัตโนมัติ

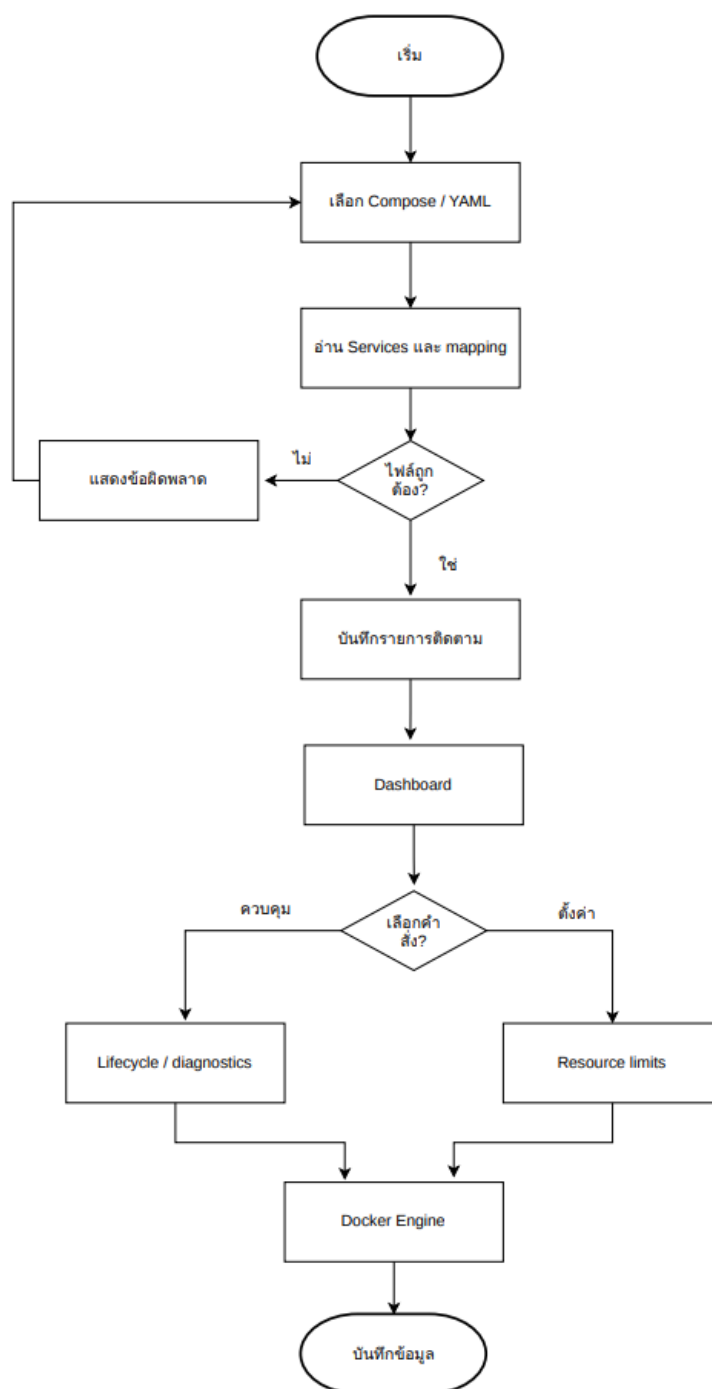
3.7 เว็บแดชบอร์ดและการเชื่อมต่อบนระบบ

3.8 การออกแบบหน้า Interface ระบบ

3.9 การทดสอบและประเมินผล

3.1 ลำดับขั้นตอนการดำเนินโครงการ

การดำเนินโครงการแบ่งออกเป็นขั้นตอนหลัก โดยแต่ละขั้นตอนเชื่อมโยงกันเพื่อให้การพัฒนากระบวนการเป็นไปตามขอบเขตที่กำหนด



3.2 การศึกษาและรวบรวมข้อมูล

การพัฒนาระบบเริ่มต้นจากการศึกษาแนวคิดและเทคโนโลยีที่เกี่ยวข้องกับการบริหารจัดการเครื่องแม่ข่าย เพื่อนำมาวิเคราะห์ความต้องการและกำหนดขอบเขตการทำงานของระบบ ดังนี้

3.2.1 แนวคิดและเทคโนโลยีที่ศึกษา

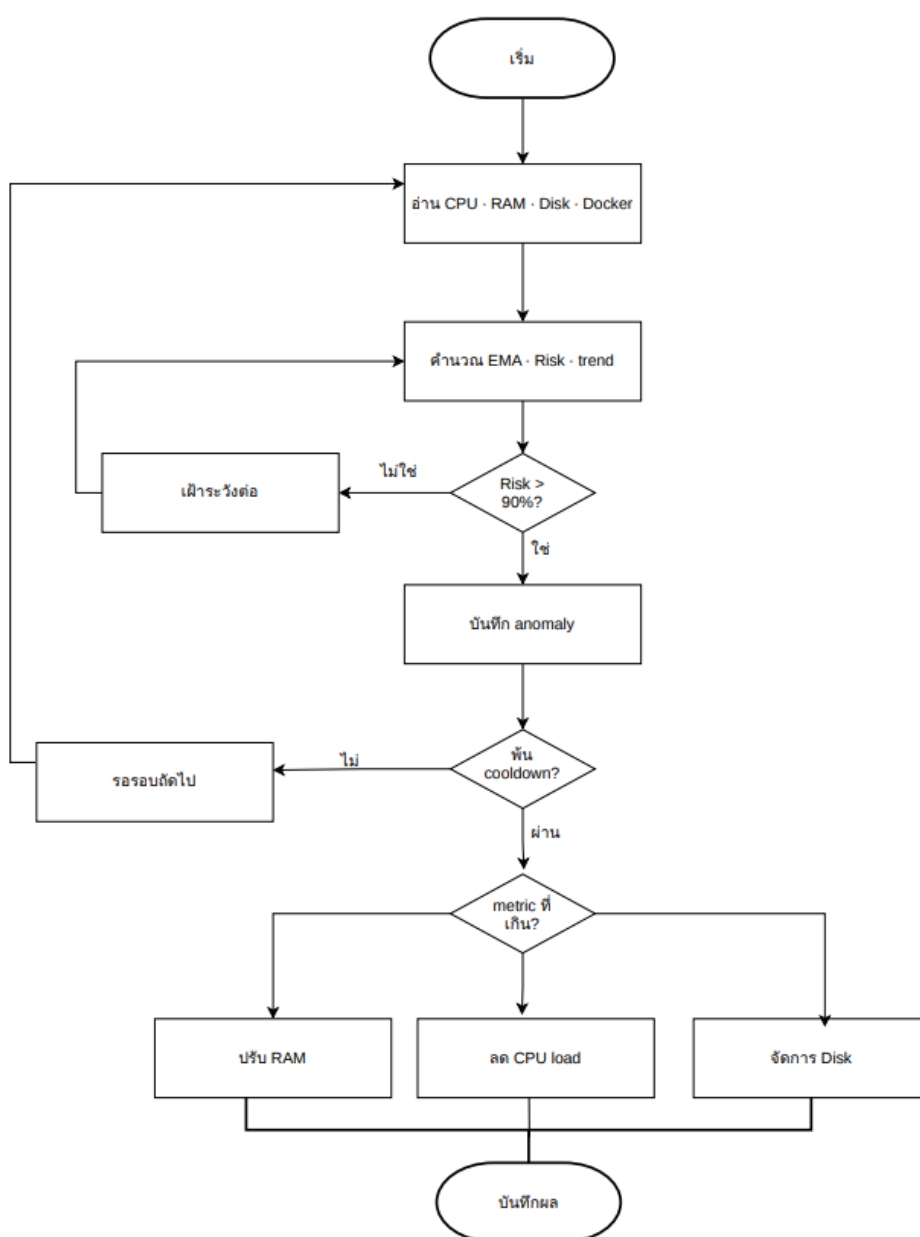
- Resource Monitoring: การอ่านค่าการใช้งาน CPU, RAM และ Disk แบบเรียลไทม์
- EMA & Hysteresis: การประเมินสถานะด้วย EMA เพื่อลดค่าผันผวน และใช้ Hysteresis ป้องกันแจ้งเตือนซ้ำซ้อน
- Docker Management: การใช้ Docker API ในการสั่ง Start, Stop และ Restart Container
- Data & Web Interface: การบันทึกประวัติลงไฟล์ CSV และการพัฒนาหน้า Web Dashboard สำหรับแสดงผลและสั่งการ

3.2.2 การวิเคราะห์ความต้องการและขอบเขตระบบ

นำข้อมูลที่ได้มาระบุความสามารถหลักของระบบ ทั้งการตรวจวัดทรัพยากรและประเมินระดับความเสี่ยง (Normal, Warning, Critical) การส่งสัญญาณแจ้งเตือนและสั่งการ Docker Container รวมถึงการบันทึกประวัติการใช้งานย้อนหลังลงไฟล์ CSV เพื่อใช้วิเคราะห์แนวโน้มระบบ

3.3 การวิเคราะห์และออกแบบระบบ

ระบบถูกออกแบบให้โปรแกรมเดสก์ท็อปเป็นจุดประสานงานหลัก โดยแบ่งการทำงานออกเป็นโมดูลตามหน้าที่ ได้แก่ การอ่านทรัพยากร การประเมินสถานะ การจัดการ Docker การแก้ไขปัญหา การให้บริการเว็บ และการสำรองข้อมูล การแยกโมดูลช่วยลดความซับซ้อนและทำให้แต่ละส่วนสามารถปรับปรุงหรือบำรุงรักษาได้ง่ายขึ้น



3.3.1 ฝั่งการทำงานของระบบ

เมื่อเริ่มโปรแกรม ระบบจะโหลดค่าการตั้งค่า เตรียมส่วนติดต่อผู้ใช้ และเริ่มกระบวนการอ่าน Metrics จากนั้น collector จะอ่านค่าทรัพยากรและบันทึกลง CSV ก่อนส่งข้อมูลให้ brain ประเมินค่า EMA, Risk Score และสถานะของระบบ เมื่อเข้าเงื่อนไขจะแจ้งเตือนหรือดำเนินการตามที่กำหนด ส่วนข้อมูลสถานะจะถูกส่งไปแสดงผลบนหน้าเว็บและส่วนติดต่อผู้ใช้

ลำดับการทำงานหลัก

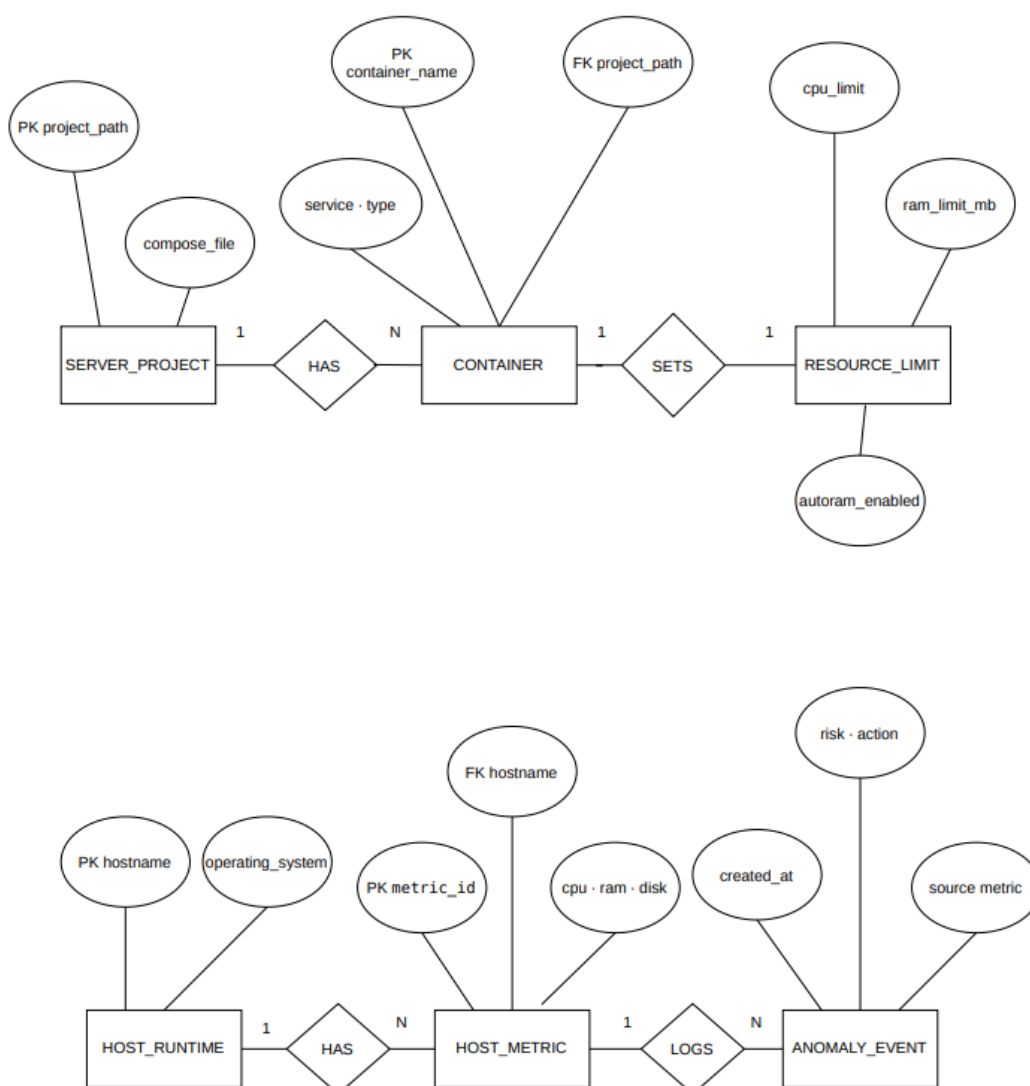
1. เริ่มโปรแกรมและโหลดการตั้งค่า
2. อ่าน CPU, RAM และ Disk
3. บันทึกข้อมูลลง CSV
4. คำนวณ EMA และ Risk Score
5. ตรวจสอบ Threshold และ Cooldown
6. เปลี่ยนสถานะหรือแจ้งเตือนเมื่อเข้าเงื่อนไข
7. ดำเนินการ Remediation หากกำหนดไว้
8. อัปเดตข้อมูลบนหน้าจอและเว็บ Dashboard

3.3.2 Context Diagram

Context Diagram แสดงความสัมพันธ์ระหว่างระบบกับองค์ประกอบภายนอก โดยระบบรับข้อมูลทรัพยากรและคำสั่งจากผู้ใช้ เชื่อมต่อ Docker เพื่ออ่านและจัดการ Container และให้บริการข้อมูลผ่าน Web Dashboard รวมถึงสามารถเชื่อมต่อบริการที่เกี่ยวข้องกับการแจ้งเตือนและการสำรองข้อมูลตามการตั้งค่าของระบบ

3.3.3 ER Diagram

แผนภาพ ER แสดงโครงสร้างข้อมูลและความสัมพันธ์ของระบบ โดยส่วนจัดการเครื่องแม่ข่ายประกอบด้วย SERVER_PROJECT, CONTAINER และ RESOURCE_LIMIT ซึ่งเครื่องแม่ข่ายหนึ่งโครงการมีคอนเทนเนอร์ได้หลายรายการ และคอนเทนเนอร์แต่ละรายการกำหนดขีดจำกัดการใช้ทรัพยากร เช่น CPU และ RAM ได้ นอกจากนี้ยังมีส่วนติดตามการทำงานของเครื่อง ได้แก่ HOST_RUNTIME, HOST_METRIC และ ANOMALY_EVENT โดยเครื่องหนึ่งสามารถมีข้อมูลตัวชี้วัดหลายรายการ และข้อมูลเหล่านี้สามารถบันทึกเหตุการณ์ผิดปกติได้หลายเหตุการณ์ เพื่อใช้ติดตามและวิเคราะห์การทำงานของระบบ



3.3.4 Data Dictionary

Data Dictionary ใช้อธิบายข้อมูลสำคัญที่ระบบจัดเก็บและนำไปใช้ในกระบวนการทำงาน โดยแบ่งตามรูปแบบไฟล์และหน้าที่ ดังนี้

ไฟล์/รูปแบบ	ข้อมูล	หน้าที่
metrics_db.csv	Timestamp, CPU, RAM, Disk	เก็บข้อมูล Metrics และประวัติ
resource_limits.json	CPU/RAM ของ Container	เก็บ Resource Limits
alert_thresholds.json	ค่า Threshold	ใช้กำหนดเงื่อนไขการแจ้งเตือน
tracked_containers.json	รายการ Container	ใช้กำหนดรายการที่ติดตาม
app_preferences.json	Theme, UI, Backup	เก็บค่าการตั้งค่าผู้ใช้
notifier_config.json	Popup และ Cooldown	กำหนดการแจ้งเตือน
email_config.json	SMTP Host, Port และ Credential	กำหนดค่าการส่งอีเมล

3.4 หลักการทำงานของโปรแกรม

กระบวนการทำงานของโปรแกรมเริ่มจากการอ่านค่าทรัพยากรของเครื่องด้วย psutil ได้แก่ CPU, RAM และ Disk แล้วบันทึกข้อมูลพร้อมเวลาใน CSV จากนั้นตัวประเมินจะคำนวณ EMA แยกตามทรัพยากรและใช้ค่าที่สูงที่สุดเป็น Risk Score

ระบบใช้ Hysteresis เพื่อควบคุมการเปลี่ยนสถานะ โดยใช้ค่า Threshold สำหรับเข้าสู่สถานะผิดปกติและค่าที่ต่ำกว่าสำหรับกลับเข้าสู่สถานะปกติ เพื่อลดการเปลี่ยนสถานะไปมาเมื่อค่าทรัพยากรมีความผันผวน

ในส่วนของ Docker โปรแกรมสามารถอ่านสถานะ Container อ่าน Logs ส่งคำสั่ง และกำหนด Resource Limits โดยคำสั่งต่าง ๆ มีการกำหนด Timeout เพื่อป้องกันการทำงานค้าง นอกจากนี้ยังมีการปรับ RAM อัตโนมัติสำหรับ Container ตามกฎที่กำหนดไว้

3.5 การจัดการข้อมูลและการจัดเก็บ Metrics

ระบบบันทึกค่าทรัพยากร CPU, RAM และ Disk พร้อม Timestamp ลงในไฟล์ CSV เพื่อใช้แสดงข้อมูลย้อนหลังและสร้างกราฟ ส่วนข้อมูลการตั้งค่าและการทำงานของระบบถูกแยกจัดเก็บในไฟล์ JSON ตามหน้าที่ เช่น Resource Limits, Alert Thresholds, Tracked Containers และ User Preferences

ระบบยังมีการสำรองข้อมูลด้วยการสร้างไฟล์ ZIP จากแหล่งข้อมูลที่กำหนด โดยสร้างไฟล์ชั่วคราวก่อนแทนที่ไฟล์เดิม และสามารถกำหนดให้ทำงานทันทีและทำซ้ำเป็นระยะ

3.6 การจัดการ Docker และการทำงานอัตโนมัติ

ผู้ใช้สามารถเพิ่มไฟล์ Docker Compose เพื่อให้โปรแกรมอ่านรายการบริการและเชื่อมโยงกับ Container ที่มีอยู่ ระบบรองรับการ Start, Stop, Restart, อ่าน Logs และส่งคำสั่งภายใน Container รวมถึงการกำหนด CPU และ RAM Limit

เมื่อระบบตรวจพบสถานะที่เข้าเงื่อนไข สามารถดำเนินการ Remediation บางประเภท เช่น การจัดการ Cache ไฟล์ชั่วคราว Process หรือ Container โดยมีการกำหนด Cooldown เพื่อจำกัดการเรียกคำสั่งซ้ำ อย่างไรก็ตาม การดำเนินการดังกล่าวอาจส่งผลกระทบต่อบริการ จึงควรตรวจสอบเงื่อนไขก่อนเปิดใช้งานกับระบบจริง

3.7 เว็บแดชบอร์ดและการเชื่อมต่อระบบ

เว็บเซิร์ฟเวอร์ทำงานอยู่เบื้องหลังและให้บริการ Dashboard ผ่านเครือข่าย LAN โดยใช้ Flask ในการจัดการหน้าเว็บและ API หากเปิดใช้งาน Cloudflare Tunnel ระบบสามารถติดตาม Container ที่เกี่ยวข้องและแสดง URL สำหรับเข้าถึง Dashboard

API สำหรับอ่านสถานะ Container การสั่งงาน การตั้ง Resource Limit และการอ่าน Logs จะตรวจสอบ Session ก่อนดำเนินการ เพื่อจำกัดการเข้าถึงฟังก์ชันที่เกี่ยวข้องกับการจัดการระบบ

3.8 การออกแบบหน้า Interface ระบบ

ส่วนติดต่อผู้ใช้ถูกออกแบบเพื่อช่วยให้ผู้ดูแลสามารถตรวจสอบสถานะทรัพยากร CPU, RAM, Disk รายการ Container และกราฟข้อมูลย้อนหลังจากจุดเดียวหน้า Dashboard จะแสดงข้อมูลสถานะล่าสุดพร้อมรายการเหตุการณ์ระหว่างการทำงาน เพื่อให้ผู้ดูแลระบบสามารถติดตามความผิดปกติได้อย่างทันท่วงที ส่วนการจัดการ Container จะแสดงรายการบริการที่ถูกเลือกติดตาม พร้อมปุ่มคำสั่งสำหรับควบคุม เปลี่ยนสถานะ และตรวจสอบรายละเอียดที่เกี่ยวข้องนอกจากนี้ยังจัดวางเลย์เอาต์ให้รองรับการแสดงผลบนอุปกรณ์ที่หลากหลาย เพื่อความสะดวกในการใช้งานผ่านเว็บเบราว์เซอร์ทุกรูปแบบรวมถึงมีการออกแบบระบบแจ้งเตือนแบบเรียลไทม์บนหน้าจอ เพื่อให้ผู้ดูแลรับทราบการเปลี่ยนแปลงสถานะความเสี่ยงได้ในทันที

3.9 การทดสอบและประเมินผล

การทดสอบแบ่งเป็นการตรวจสอบการทำงานของแต่ละโมดูลและการทำงานร่วมกัน ทั้งการอ่าน Metrics, การคำนวณสถานะ, การควบคุม Docker และ Web Interface ผลการทดสอบจำแนกความสามารถที่มีอยู่จริงในซอร์สโค้ด ออกจากผลการทดสอบแพ็คเกจและการใช้งานจริงอย่างชัดเจน เพื่อให้ได้ข้อสรุปที่ถูกต้องการประเมินผลเน้นความแม่นยำในการแจ้งเตือน ความเสถียรในการสั่งการ Container และความสมบูรณ์ในการบันทึกข้อมูลย้อนหลังลงไฟล์ CSV รวมถึงมีการจำลองสภาวะภาระงานสูง (Load Test) เพื่อทดสอบการทำงานของอัลกอริทึม EMA และ Hysteresis ในสถานะจริง

บทที่ 4

ผลการพัฒนาและการทดสอบ

บทนี้นำเสนอผลการพัฒนาโปรแกรม ผลการทดสอบเชิงหน้าที่และ self-test ของ Windows EXE พร้อมอภิปรายผลและข้อจำกัดของหลักฐานที่มี เพื่อแสดงให้เห็นขอบเขตความสามารถของระบบและประเด็นที่ควรพิจารณาในการใช้งานต่อไป

4.1 ผลการพัฒนา

4.2 หน้าจอของโปรแกรม

4.3 รายงานผลทดสอบเชิงหน้าที่

4.4 ผลทดสอบของ Windows EXE

4.5 อภิปรายผล

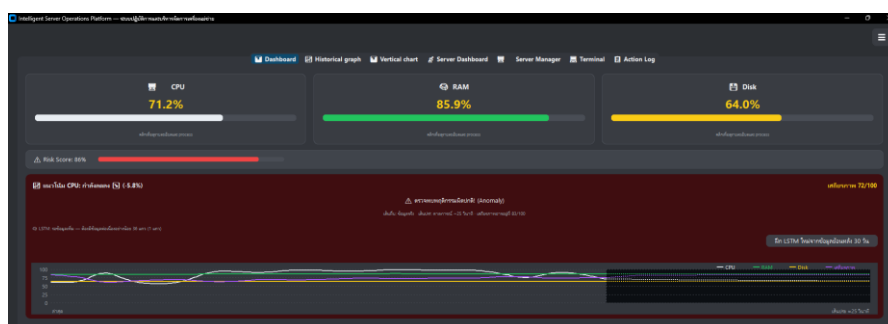
4.6 ข้อจำกัดของผลทดสอบ

4.1 ผลการพัฒนา

ระบบที่พัฒนาขึ้นเป็นโปรแกรมเดสก์ท็อปสำหรับติดตามทรัพยากร ดูประวัติการใช้งาน จัดการ Docker container ตั้งค่าขีดจำกัดทรัพยากร และแสดงการแจ้งเตือน นอกจากนี้ยังมีฟังก์ชันสำรองข้อมูลและเมนูเปิดเว็บแดชบอร์ด โดยข้อมูลทรัพยากรบันทึกเป็นไฟล์ CSV ส่วน Action Log และ web events ใช้แสดงเหตุการณ์ระหว่างเปิดโปรแกรม

4.2 หน้าจอของโปรแกรม

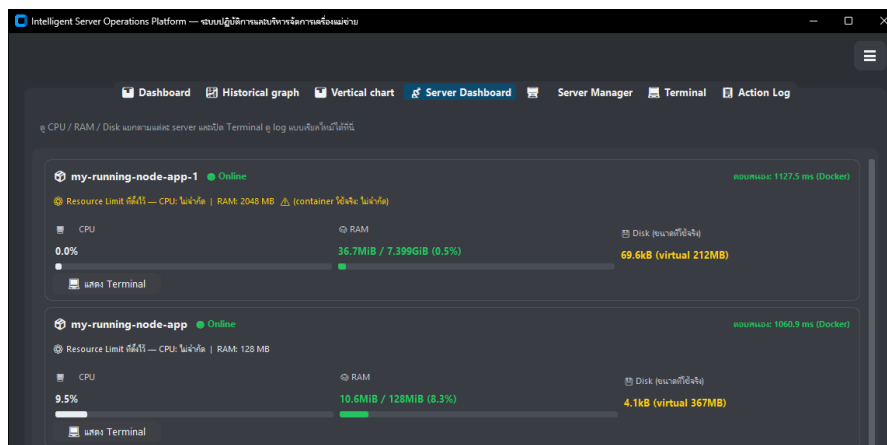
ภาพหน้าจอต่อไปนี้นำมาจากเอกสารผลทดสอบที่ได้รับ ใช้ประกอบการอธิบายหน้าตาและเมนูของโปรแกรมเท่านั้น ไม่ได้ยืนยันว่าทุกฟังก์ชันผ่านการทดสอบในรุ่นปัจจุบัน



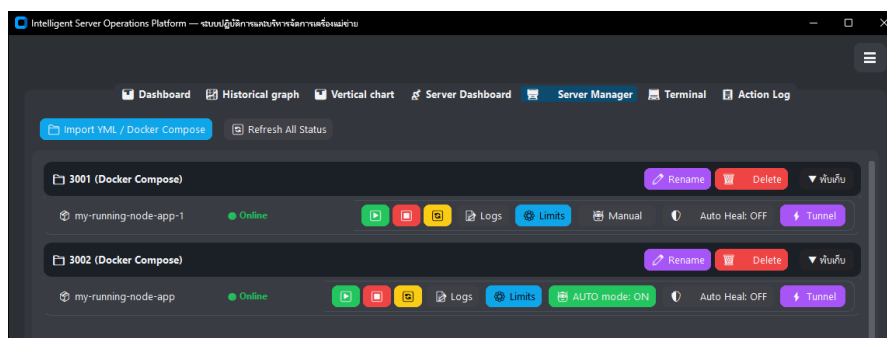
รูปที่ 4.1 หน้าหลักสำหรับติดตามทรัพยากร



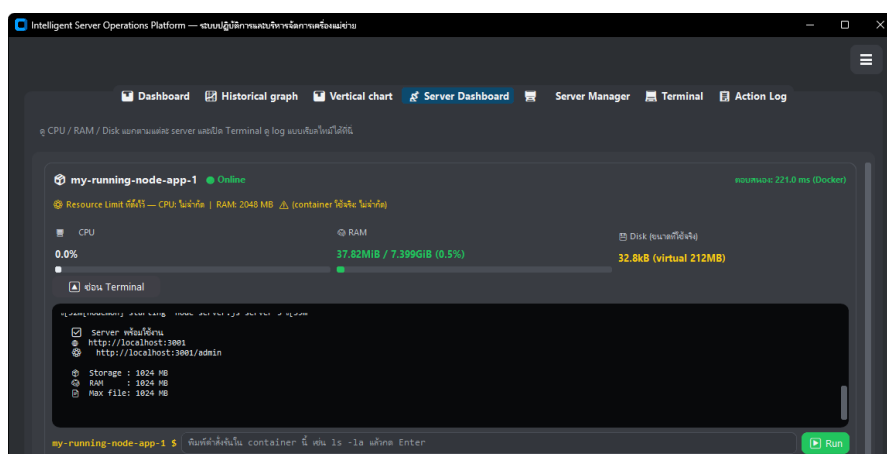
รูปที่ 4.2 หน้าประวัติและกราฟการใช้ทรัพยากร



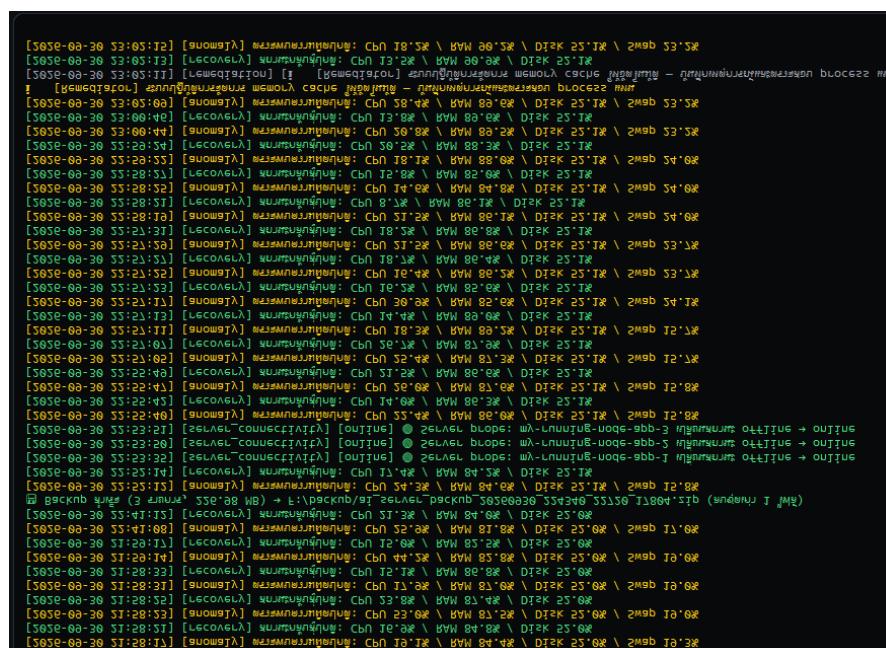
รูปที่ 4.3 หน้า Server Dashboard



รูปที่ 4.4 หน้า Server Manager



รูปที่ 4.5 หน้าต่าง Terminal สำหรับส่งคำสั่งภายใน container



รูปที่ 4.6 หน้า Action Log สำหรับแสดงเหตุการณ์ระหว่าง session

4.3 รายงานผลทดสอบเชิงหน้าที่

เอกสารทดสอบที่ได้รับรายงานผลการทดสอบแบบ black-box จำนวน 10 กรณีว่าผ่านทั้งหมด ครอบคลุมการอ่านทรัพยากร การตรวจจับตามเกณฑ์ การจัดการ container การเข้าใช้งาน dashboard การสร้างไฟล์สำรอง การดูกราฟ และการใช้ terminal อย่างไรก็ตาม เอกสารไม่ได้แนบข้อมูลดิบหรือรายละเอียดสภาพแวดล้อมครบถ้วน ผลที่นำเสนอจึงเป็นไปตามรายงานต้นทางและไม่ได้ทดสอบซ้ำในรายงานฉบับนี้

Average historical data / ข้อมูลสรุปตามช่วงเวลา							
ช่วงข้อมูล: ภาพรวมทั้งหมด - 2,892 ตัวอย่างอื่น ดูเพิ่ม 5 ช่วง							
ช่วงเวลา	จำนวน	CPU แฉก	CPU คัด	RAM แฉก	RAM คัด	Disk แฉก	Disk คัด
2020-09-25	510	8.0%	34.3%	75.6%	79.2%	50.2%	50.3%
2020-09-27	516	14.2%	37.2%	81.5%	86.5%	49.9%	49.9%
2020-09-28	114	13.5%	25.2%	76.5%	79.3%	50.3%	50.3%
2020-09-29	45	11.5%	21.5%	83.9%	93.1%	50.2%	50.4%
2020-09-30	1,707	10.9%	70.7%	83.9%	99.1%	52.7%	54.5%

รูปที่ 4.7 ข้อมูลสรุปตามช่วงเวลา

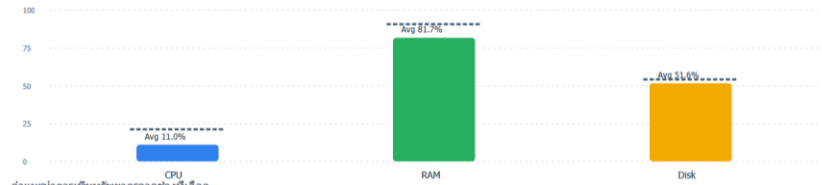
Resource Performance Report / รายงานการใช้ทรัพยากร

ช่วงข้อมูล: ภาพรวมทั้งหมด

สร้างเมื่อ 2026-09-30 23:12:24 จำนวนตัวอย่าง 2,892 ตั้งแต่ 2026-09-25 13:36:17 ถึง 2026-09-30 21:43:32

ทรัพยากร	ต่ำสุด	เฉลี่ย	P95	สูงสุด
CPU	1.7%	11.0%	21.2%	70.7%
RAM	61.9%	81.7%	90.7%	99.1%
Disk	49.7%	51.6%	54.2%	54.5%

Resource average / P95



คำแนะนำการเพิ่มทรัพยากรจากช่วงที่เลือก

CPU: ตัวโหนดเฉลี่ย ถึงจุด 16 cores - P95 21.2%

RAM: +7 GB RAM ถึงจุด 15.7 GB - P95 90.7% - รวมแนะนำ 23 GB

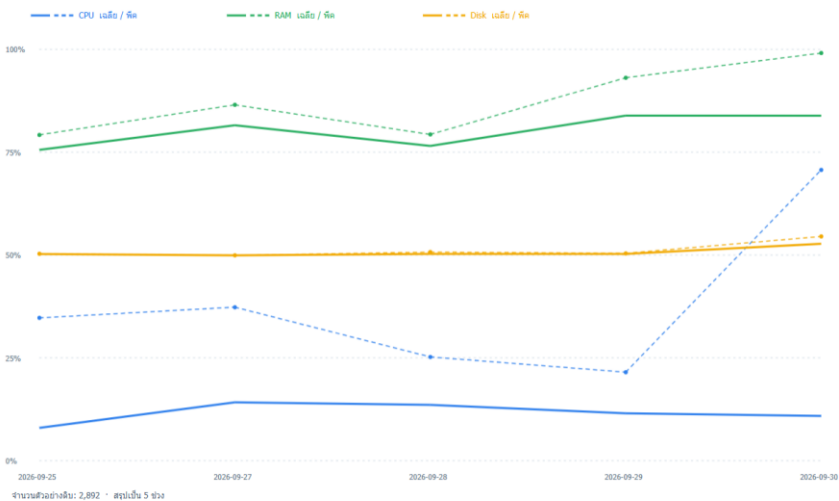
Disk: ตัวโหนดเฉลี่ย พาคู่ที่ขึ้นระบบ 275.1 GB - P95 54.2%

คำนวณจากจำนวน P95 และเฉลี่ยภายใน 20% เพื่อเป็นข้อมูลประกอบการวางแผน ไม่ได้นำมาใช้ในการตัดสินใจเพิ่มทรัพยากร
พิกัดที่ไม่เป็นกราฟแสดงไว้มาจาก Historical Graph และจากค่าเฉลี่ยที่สรุปจากข้อมูลอื่น

รูปที่ 4.8 รายงานการใช้ทรัพยากร

Historical Graph / แนวโน้มการใช้ทรัพยากร

ช่วงข้อมูล: ภาพรวมทั้งหมด - สรุปค่าเฉลี่ย/กราฟ และค่าสูงสุด



จำนวนตัวอย่างเฉลี่ย: 2,892 - สรุปเป็น 5 ช่วง

Intelligent Server Operations Platform | 2/3

รูปที่ 4.9 แนวโน้มการใช้ทรัพยากร

4.4 ผลทดสอบของ Windows EXE

รายงาน self-test ระบุว่าโปรแกรม Windows EXE ผ่านการตรวจสอบ 7 รายการ ได้แก่ การโหลด Tcl/Tk และทรัพยากรที่ฝังในไฟล์ การใช้ธีม Dark/Gray การอ่านไฟล์ Docker Compose การทดสอบหน้าเว็บด้วย Flask test client การอ่านและเขียน app-data และการเปิดโปรแกรมซ้ำแบบ one-file relaunch การทดสอบนี้ตรวจสอบองค์ประกอบของโปรแกรมและแพ็คเกจ ไม่ได้ประเมินประสิทธิภาพการทำงานกับ container จริงหรือการใช้งานต่อเนื่องระยะยาว

```
=====
RUNNING FROZEN EXE SELF-TEST
=====

=== Intelligent Server Operations Platform self-test ===
[PASS] Tcl/Tk: OK
[PASS] Bundled resources: OK
[PASS] All UI themes: OK
[PASS] Docker Compose YAML: OK
[PASS] Web dashboard without login: OK
[PASS] Writable app-data: OK
[PASS] One-file relaunch: OK
=== SELF-TEST PASS ===
```

รูปที่ 4.10 Self-test

4.5 อภิปรายผล

ผลที่มีแสดงว่าโปรแกรมรวมฟังก์ชันหลักไว้ในระบบเดียว และรายงานที่ได้รับระบุว่ากรณีทดสอบเชิงหน้าที่ผ่าน ส่วน self-test แสดงว่าองค์ประกอบสำคัญของ Windows EXE ทำงานได้ตามรายการที่ตรวจสอบ แต่ยังไม่มีความเพียงพอสำหรับสรุปความแม่นยำในการตรวจจับหรือประสิทธิภาพในสภาพแวดล้อมที่หลากหลาย อีกทั้งยังไม่มีผลทดสอบการกู้คืนข้อมูลจากไฟล์สำรอง

4.6 ข้อจำกัดของผลทดสอบ

ผลทดสอบที่มีไม่ได้ระบุประเภทเครื่องและรุ่นซอฟต์แวร์อย่างครบถ้วน และไม่มีข้อมูล metrics, เวลาตอบสนอง หรือหลักฐานอธิบายกรณี นอกจากนี้ยังไม่มีผลทดสอบการกู้คืนไฟล์สำรอง การทำงานต่อเนื่อง 8 ชั่วโมง และการเข้าถึง API โดยไม่เข้าสู่ระบบ ผลการตั้งธีม Light ในเอกสารเดิมไม่ตรงกับรุ่นปัจจุบัน ซึ่งรองรับธีม Dark และ Gray จึงควรทดสอบใหม่ก่อนนำผลดังกล่าวไปอ้างอิง

บทที่ 5

สรุปผลและข้อเสนอแนะ

5.1 สรุปผล

โครงการพัฒนาเครื่องมือสำหรับดูการใช้ CPU, RAM และ disk ของเครื่องที่เปิดโปรแกรม พร้อมจัดการ Docker container ที่เลือกจาก Docker Compose ระบบใช้ EMA และ threshold แบบ hysteresis เพื่อแสดงสถานะตามกฎ มีเว็บแดชบอร์ดที่ต้องเข้าสู่ระบบ บันทึก metrics ลง CSV และสำรองข้อมูลที่กำหนดเป็น ZIP โปรแกรมรุ่นส่งมอบถูกจัดเป็น Windows EXE แบบ one-file และมี self-test สำหรับตรวจ runtime

หลักฐานที่แนบมาสรุปผล black-box 10 รายการและ self-test 7

รายการว่าผ่านตามขอบเขตของแต่ละรายงาน

ข้อมูลยังไม่เพียงพอสำหรับสรุปประสิทธิภาพเชิงปริมาณ ความทนทานระยะยาว

ความแม่นยำในการตรวจจับ หรือผลต่อ downtime

5.2 ข้อจำกัดของระบบ

- เกณฑ์และน้ำหนัก EMA ยังเป็นค่าคงที่ ไม่ได้ปรับตามชนิดงานหรือ workload
 - disk ที่อ่านเป็นพาร์ติชันระบบหลัก และการจัดการ container ต้องมี Docker ใช้งานได้
 - Quick Tunnel เป็นช่องทางสำหรับทดลอง
- การเปิดใช้จริงจากภายนอกต้องจัดการความปลอดภัยและรูปแบบ tunnel ให้เหมาะสม
- ระบบสำรองเก็บ ZIP ชุดล่าสุดตามรายการแหล่งข้อมูล ไม่รวม host และ Docker volumes โดยอัตโนมัติ และยังไม่มีการทดสอบกู้คืน
 - การแก้ไข process หรือไฟล์อัตโนมัติอาจมีผลต่อบริการ; ควรจำกัดสิทธิ์และทดสอบก่อนใช้งานจริง
 - ยังไม่พบการจัดเก็บ Action Log/web events ถาวรในโค้ดที่ตรวจ

5.3 ข้อเสนอแนะสำหรับการพัฒนาต่อ

- ทำชุดทดสอบซ้ำได้ โดยบันทึกสภาพแวดล้อม input, ผลที่คาดหวัง, ผลจริง และเวลาที่ใช้ในแต่ละกรณี
- ทดสอบ backup ด้วยการ restore ไปยังตำแหน่งแยก และตรวจไฟล์สำคัญหลังการกู้คืน
- เพิ่มตัวเลือกเกณฑ์รายการเครื่องหรือรายการ container พร้อมรายการยกเว้นสำหรับ process ที่ห้ามจัดการ
- เพิ่มการเก็บ event log แบบถาวรโดยหลีกเลี่ยงรหัสผ่าน token และข้อมูลอ่อนไหว
- ทบทวนการยืนยันตัวตน สิทธิ API การจัดการ secret และการเชื่อมต่อ HTTPS ก่อนเปิดใช้ภายนอก
- ประเมินค่า EMA และ threshold ด้วย workload หลากหลายรูปแบบ ก่อนกล่าวถึงความแม่นยำหรือการลดเวลาหยุดให้บริการ

5.4 งานที่ควรทำก่อนส่งฉบับสมบูรณ์

ให้ผู้จัดทำตรวจสอบชื่อเรื่อง ชื่อบุคคล และรูปแบบชื่อหลักสูตรตามเอกสารทางการ
เพิ่มหลักฐานจากการทดสอบรุ่นล่าสุด และใช้แบบฟอร์มอนุมัติของหลักสูตรก่อนพิมพ์ฉบับส่ง
ส่วนข้อมูลส่วนตัวที่ไม่จำเป็นต่อการประเมินไม่ควรใส่ในเล่ม

บรรณานุกรม

- [1] psutil developers. (ม.ป.ป.). psutil documentation. สืบค้นเมื่อ 26 กันยายน 2569, จาก <https://psutil.readthedocs.io/stable/>
- [2] Docker, Inc. (ม.ป.ป.). Resource constraints. Docker Documentation. สืบค้นเมื่อ 26 กันยายน 2569, จาก https://docs.docker.com/engine/containers/resource_constraints/
- [3] Pallets Projects. (ม.ป.ป.). Quickstart — Flask Documentation. สืบค้นเมื่อ 26 กันยายน 2569, จาก <https://flask.palletsprojects.com/en/stable/quickstart/>
- [4] Cloudflare. (2026, 20 เมษายน). Quick Tunnels. Cloudflare One Documentation. <https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/do-more-with-tunnels/trycloudflare/>
- [5] Python Software Foundation. (ม.ป.ป.). csv — CSV File Reading and Writing. Python Documentation. <https://docs.python.org/3/library/csv.html>
- [6] PyInstaller Development Team. (ม.ป.ป.). What PyInstaller Does and How It Does It. PyInstaller Documentation. <https://pyinstaller.org/en/stable/operating-mode.html>
- [7] Tom Schimansky. (ม.ป.ป.). Documentation Introduction. CustomTkinter Documentation. <https://customtkinter.tomschimansky.com/documentation/>
- [8] pandas development team. (ม.ป.ป.). pandas.read_csv. pandas documentation. https://pandas.pydata.org/pandas-docs/stable/reference/api/pandas.read_csv.html
- [9] NIST/SEMATECH. (ม.ป.ป.). What is Exponential Smoothing? e-Handbook of Statistical Methods. <https://www.itl.nist.gov/div898/handbook/pmc/section4/pmc43.htm>

[10] Beyer, B., Jones, C., Petoff, J., & Murphy, N. R. (บ.ก.). (2016). Monitoring Distributed Systems. ใน Site Reliability Engineering. Google. <https://sre.google/sre-book/monitoring-distributed-systems/>

[11] Notaro, P., Cardoso, J., & Gerndt, M. (2020). A Systematic Mapping Study in AIOps. arXiv:2012.09108. <https://arxiv.org/abs/2012.09108>

[12] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008). Isolation Forest. 2008 Eighth IEEE International Conference on Data Mining. <https://doi.org/10.1109/ICDM.2008.17>

[13] scikit-learn developers. (ม.ป.ป.). Model evaluation: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html

หมายเหตุ: บรรณานุกรมนี้ครอบคลุมเอกสารทางเทคนิคที่ใช้อธิบายเครื่องมือในระบบ
ควรเพิ่มงานวิจัยที่เกี่ยวข้องตามที่อาจารย์ที่ปรึกษาหรือคู่มือหลักสูตรกำหนด

ภาคผนวก ก

คู่มือการใช้งานระบบปฏิบัติการและบริหารจัดการเครื่องแม่ข่าย

แนวทางการใช้งาน

1. เตรียมเครื่อง

ติดตั้งและเปิด Docker Engine/Desktop หากต้องการจัดการ container ตรวจสอบว่า Docker daemon พร้อมใช้งานและบัญชีผู้ใช้มีสิทธิ์เรียก Docker CLI

2. เปิดโปรแกรม

เปิดไฟล์ EXE จากนั้นตรวจหน้า dashboard ว่าค่า CPU, RAM และ disk ปรากฏตามเครื่องที่กำลังใช้งาน

3. เพิ่มบริการ

เลือกไฟล์ Docker Compose YAML ที่ต้องการให้โปรแกรมอ่าน ตรวจสอบชื่อบริการและ container ก่อนสั่งงาน

4. จัดการ container

ตรวจสอบสถานะก่อน แล้วเลือก start, stop หรือ restart ตามความจำเป็น คำสั่ง terminal จะทำงานภายใน container ที่เลือก

5. ใช้เว็บแดชบอร์ด

เปิดเว็บจากลิงก์ LAN หรือ Quick Tunnel ที่โปรแกรมแสดง
เข้าสู่ระบบก่อนดูสถานะหรือส่งคำสั่ง

6. สำรองข้อมูล

เปิดการสำรอง เลือกตำแหน่งปลายทางที่เข้าถึงได้ ระบบทำไฟล์ ZIP

ตามรอบที่ตั้งไว้และเก็บชุดล่าสุด

ก่อนพึ่งพาข้อมูลสำรองควรทดลองแตกไฟล์และกู้คืนในโพลเดอร์ทดสอบ

ภาคผนวก ข

คำสั่งแต่งตั้งคณะกรรมการจัดทำร่างคู่มือการพิมพ์ปริญญาบัตร

คณะครุศาสตร์อุตสาหกรรม

ภาคผนวก ค

มติคณะกรรมการประจำคณะครุศาสตร์อุตสาหกรรม

ประวัติผู้เขียน

วัชรปกรณ มีชัย

ปรัชญาพงษ์ คุณมี